
Maintain Integration

Given a use case, identify the appropriate error handling, escalation and recovery procedures for a failed integration.

Table of Contents



[Business Scenario](#)

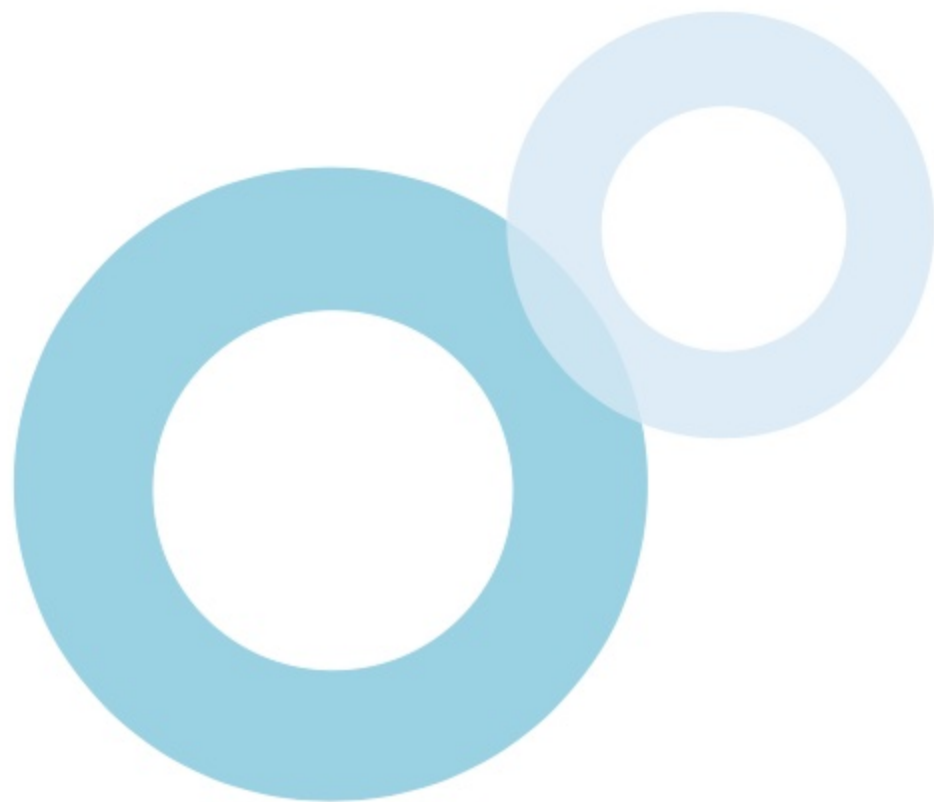


[Requirements & Solutions](#)



After studying this topic, you should be able to:

-  Identify the error handling, escalation and recovery procedures for various integration patterns.
-  Identify the error handling, escalation, and recovery strategy for various business use cases.



Introduction

The strategy used for **error handling, escalation, and recovery** in an integration solution depends on the **integration pattern**. For instance, when an integration is based on the Remote Process Invocation - Request & Reply pattern, the caller (e.g., a synchronous Apex REST callout) should manage error handling. A suitable **recovery mechanism** is also important to consider in an integration solution design. In most cases, a **custom retry mechanism** can be implemented if quality-of-service requirements dictate it.



Error Handling, Escalation, and Recovery

Remote Process Invocation (Fire & Forget)

In a remote process invocation (fire & forget) integration, the callout should handle errors in the initial invocation. The remote service should handle errors once the invocation has been handed off.

Remote Process Invocation (Request & Reply)

In a remote process invocation (request & reply) integration, the caller must manage error handling and automatic retries.



Batch Data Synchronization

In a batch data synchronization integration, the remote system must handle errors and automatic retries. A retry can be implemented for read and write operations using an ETL tool.

Remote Call-In

In a remote call-in integration, the remote system must handle errors and automatic retries, but a middleware can be used to define the logic. The custom retry mechanism must be based on idempotent design characteristics.

Data Virtualization

In a data virtualization integration, the caller should manage error handling. The Salesforce Connect Validator tool can be used to run queries and notice error types and failures causes.

Error Handling, Escalation, and Recovery

In an integration solution, the strategy used for **error handling, escalation, and recovery** depends on the **integration pattern**.

❖ REMOTE PROCESS INVOCATION - REQUEST & REPLY

When an integration is based on the Remote Process Invocation - Request & Reply pattern, the **caller** (e.g., a synchronous Apex REST callout) should manage **error handling**. **Error message** should be displayed on the **end user's page** or **logged to a table** for escalation.

As far as **recovery** is concerned, **changes should not be committed to Salesforce** until a **successful response** is received. If there's a failure, the caller can **retry** the operation.



Error Handling, Escalation, and Recovery

In an integration solution, the strategy used for **error handling, escalation, and recovery** depends on the **integration pattern**.

❖ REMOTE PROCESS INVOCATION - FIRE & FORGET

Asynchronous Apex REST or SOAP callouts should handle exceptions in the initial invocation; the **remote service** should handle exceptions once the initial invocation is handed off. The remote system can implement a **custom retry mechanism**.

For **outbound messaging**, the remote system must handle errors. Salesforce initiates retries for an outbound message if there is **no positive acknowledgement** within the **timeout period, up to 24 hours**.

For **platform events**, the **remote system** must perform error handling. Also, it can initiate **retries** for a platform event using the **Replay ID**.



Error Handling, Escalation, and Recovery

In an integration solution, the strategy used for **error handling, escalation, and recovery** depends on the **integration pattern**.

❖ BATCH DATA SYNCHRONIZATION

For **Salesforce Change Data Capture**, the **remote service** must handle **message queuing, processing, and error handling**. It must also initiate **retries** using the **Replay ID**.

For a **'read'** operation using an **ETL tool**, a **retry** should be implemented for errors that aren't infrastructure-related. If there are **repeated failures, control/error tables** should be used to **log errors, retry the operation, terminate if unsuccessful, and send notifications**. For a **'write'** operation using an **ETL tool**, the operation should be retried using the **record identifying information, success/failure notification, and a collection of errors for each record**. If there are **failed records**, an **immediate restart or subsequent job execution** can be used. If time is required to **triage and correct data**, a **delayed restart** can be used.



Error Handling, Escalation, and Recovery

In an integration solution, the strategy used for **error handling, escalation, and recovery** depends on the **integration pattern**.

❖ REMOTE CALL-IN

When an integration is based on the **Remote Call-In** pattern, the **remote system** must handle any **subsequent errors**, such as **timeouts** and the **management of retries**. A **middleware** can be used for the **error handling and recovery logic**.

For **integration recovery**, a **custom retry mechanism** based on **idempotent design characteristics** can be implemented if quality-of-service requirements dictate it.



Error Handling, Escalation, and Recovery

In an integration solution, the strategy used for **error handling, escalation, and recovery** depends on the **integration pattern**.

❖ DATA VIRTUALIZATION

For **data virtualization** solutions, the caller should manage **error handling**. When using **Salesforce Connect**, the **Salesforce Connect Validator** tool can be used to **run some common queries** and **notice error types and failure causes**.



Learn More



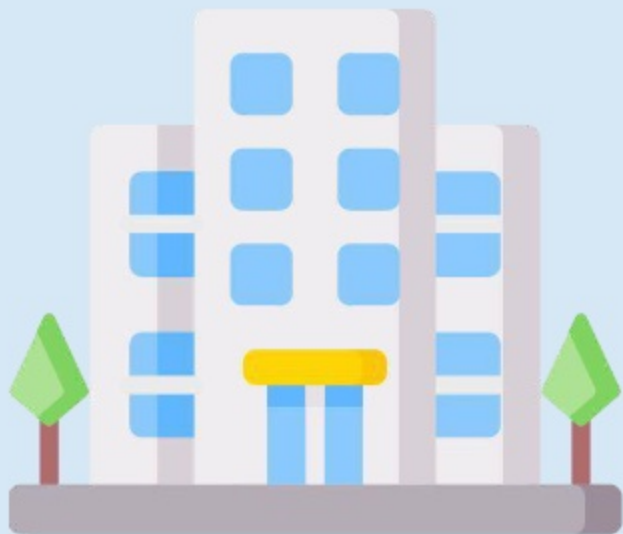
[Integration Patterns and Practices](#)

Business Scenario

Table of Contents 

Business Scenario

Read the business scenario below and determine the most appropriate solutions for the company's requirements.



Cosmic Bakery is a startup company in New York that sells various types of baked food items. It has a website that is used by hundreds of customers every day to place orders. The company also sells its products to businesses. Its sales reps use Salesforce for case and opportunity management. An ERP system is used for operations such as order processing and fulfillment. In addition, a payroll system stores the payroll data of employees.

Currently, most of the business processes in the enterprise systems are performed manually by the internal users. The company would like to integrate the systems to improve efficiency. A team of integration experts has been working on designing solutions for various integration requirements. However, since the solution design does not contain information about appropriate error handling, escalation, and recovery procedures, a solution architect has been assigned to identify them.

Requirements & Solutions

[Table of Contents](#)

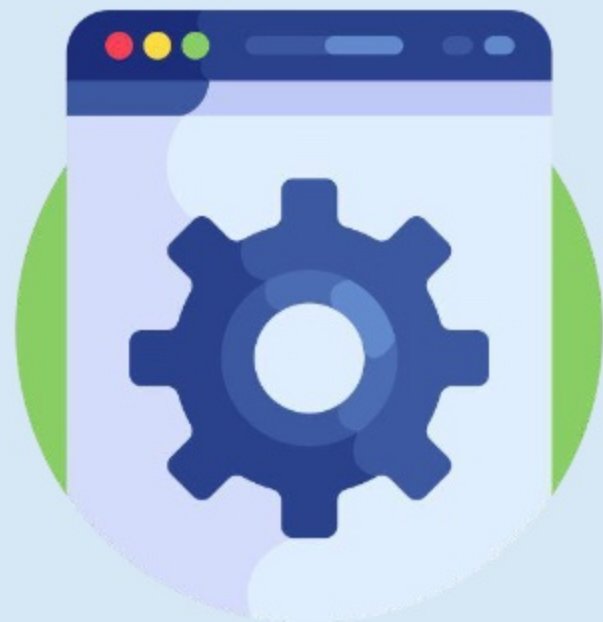


Requirement 1



SCENARIO

The sales reps use an ERP system for order management a Lightning web component in Salesforce to manage opportunities. When an opportunity reaches the 'Closed Won' stage, a synchronous Apex REST callout should send opportunity data to the ERP system for the creation of a new order record. If the order has been created successfully, the value of a custom field on the opportunity should be updated. The architect must identify any considerations for the inclusion of an error handling and recovery strategy in the integration solution.

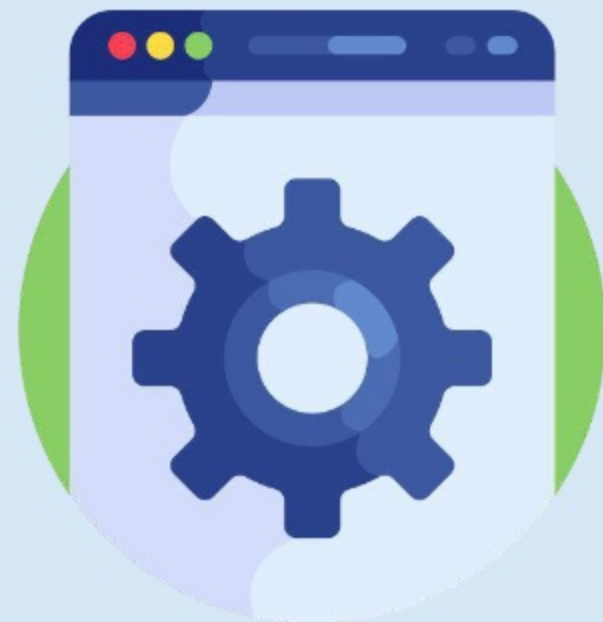


Solution 1



SOLUTION

The integration in this scenario is an example of a '**Remote Process Invocation - Request & Reply**' integration pattern. The **Lightning web component** should manage **error handling** when an error occurs. An **error message** can be displayed in the Lightning web component and also logged to a custom object for escalation. The **Apex controller** associated with the component should **retry the callout** if there is a **failure**. It is necessary to ensure that **changes** are only **committed to the database** if a **response that indicates success** is received.

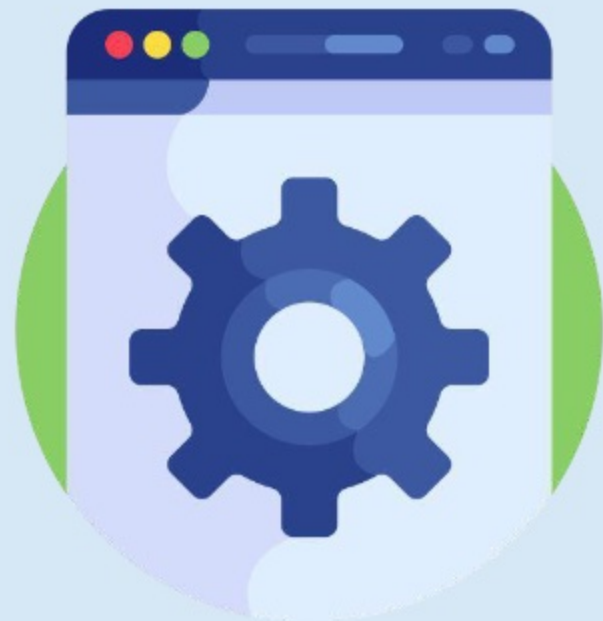


Requirement 2



SCENARIO

The support reps use Salesforce for case management. When a case is closed, a record-triggered flow should use an asynchronous Apex SOAP callout to send case data to a remote system that is mainly used for reporting and analytics. The architect needs to identify the error handling and recovery strategy that should be included in the integration solution.

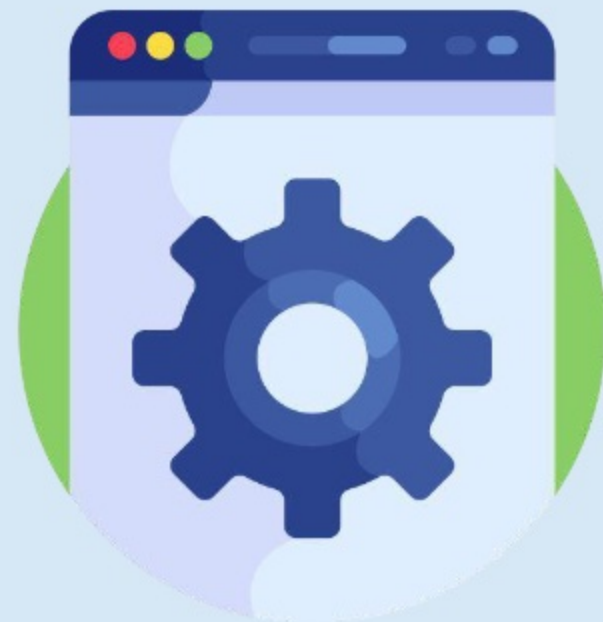


Solution 2



SOLUTION

The **Apex callout** should handle exceptions in the **initial invocation** of the external service in the remote system. The **remote system** must handle any errors once the invocation is handed off for **asynchronous processing**. A **custom retry mechanism** can be implemented in the remote system in case the asynchronous processing fails.

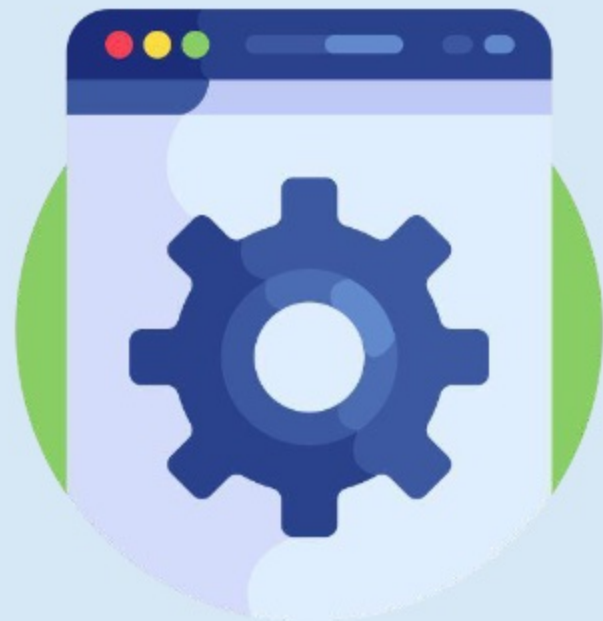


Requirement 3



SCENARIO

The company is considering the use of Salesforce Change Data Capture to synchronize account changes in Salesforce with its ERP system. The architect must identify a suitable error handling, escalation, and recovery strategy for this use case.

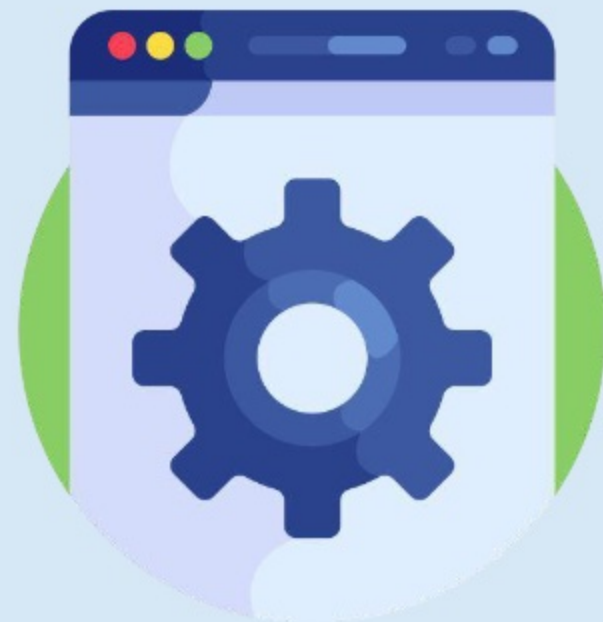


Solution 3



SOLUTION

Error handling must be performed in the ERP system since a **change event** would be handed off to it for further processing. It can handle **message queuing, processing, error handling, and logging**. The ERP system must also initiate **automatic retries** based on the quality-of-service requirements. It can use the **Replay ID** to replay the stream from a specific event, such as the last successfully captured event, and retrieve past event messages.

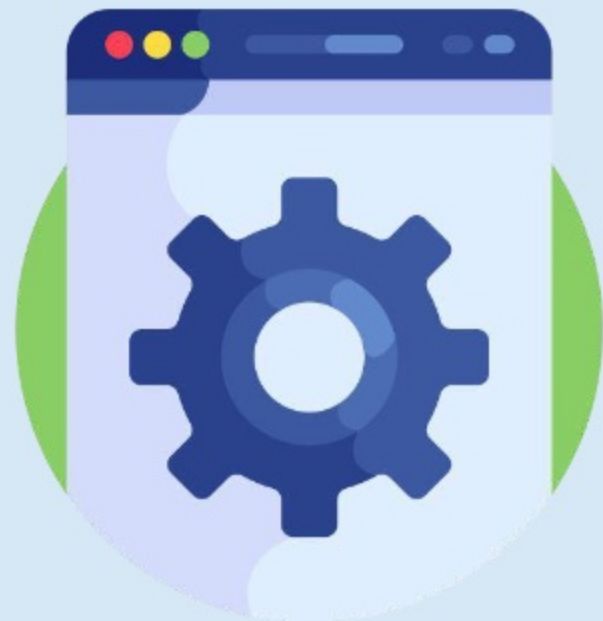


Requirement 4



SCENARIO

The company uses a homegrown payroll system to store the salary information of its employees. The HR department uses a custom app in Salesforce to manage their personal information, goals, and performance reviews. However, the HR director would also like to see their compensation details in Salesforce. An integration solution is being designed to send an employee's compensation information to Salesforce using REST API once it has been added to the payroll system. Any valid considerations related to error handling, escalation, and recovery need to be specified in the solution design.



Solution 4



SOLUTION

When using the **Remote Call-In** pattern, the remote system must be used to handle any subsequent errors, such as timeouts and the management of retries, but a middleware can be used to provide the logic for error handling and recovery. In this scenario, the **payroll system** must handle errors and the management of retries. A **middleware** can be used for the error handling and recovery logic. A **custom retry mechanism** can be created if quality-of-service requirements dictate it, but it's important to ensure **idempotent design characteristics**. The remote system must manage multiple calls if there are **errors or timeouts**. It must also implement a **reliable messaging system** to manage such scenarios.

